DEM Coaxial Fill Repository Manual

Repository guide for new graduate students

April 3, 2026

Contents

1 Purpose of This Repository **3**

2 Repository Map **3**

3 What the Solver Actually Simulates **4**

3.1 Geometry 4

3.2 Particles 4

3.3 Core Physics Model 4

4 How the Main Time Loop Works **5**

5 Composition and Packing Fraction **5**

5.1 Species Fractions 5

5.2 Important Interpretation 6

5.3 Packing Fraction Target 6

6 Important Solver Parameters **6**

6.1 Always Important 6

6.2 Composition and Size 7

6.3 Shaking and Motion 7

6.4 Contacts, Damping, and Stabilization 8

6.5 Repulsion and Settling Controls 8

6.6 Output Controls 8

7 How to Build the Solver **9**

7.1 Windows 9

7.2 Linux or macOS . . . 9

8 How to Run One Simulation **9**

8.1 Recommended First Run . . . 10

8.2 Why This Matters . . . 10

9 Understanding the Output Files **10**

9.1 XYZ Files . . . 10

9.2 VTK Files . . . 10

9.3 Optional Domain VTK . . . 11

10 Post-Processing Scripts **11**

10.1 `testing/_fill_stats.py` . . . 11

10.2 Interpretation Caution . . . 11

10.3 `testing/_atoms_plot_all.py` . . . 11

10.4 `testing_map/verify_taguchi_cases.py` . . . 11

11 Taguchi and Multi-Case Workflow **12**

11.1 Why This Exists . . . 12

11.2 Expected CSV Columns . . . 12

11.3 What the Script Produces . . . 12

11.4 How to Think About a Case Folder . . . 12

11.5 Suggested Workflow for Sweeps . . . 13

12 Old Versions and Historical Files **13**

13 Practical Advice for New Graduate Students **13**

13.1 Start Small . . . 13

13.2 Record Assumptions . . . 13

13.3 Common Beginner Mistakes . . . 13

14 Suggested First-Week Learning Plan **14**

15 Where to Edit for Common Tasks **14**

16 Known Limits of the Current Repository **14**

17 Final Takeaway **15**

1 Purpose of This Repository

This repository contains a CPU-based discrete element method (DEM)-style simulator for packing particles into a coaxial, annular column. The main use case is to study how particles with different size distributions and compositions fill and settle inside a cylindrical annulus under gravity, shaking, optional repulsion, and optional top-ram compression.

The repository is more than just the solver. It also includes:

- Windows build scripts.
- Single-run execution scripts for interactive testing.
- Python scripts for post-processing and visualization.
- A Taguchi-style parameter sweep workflow for running many cases.
- Verification scripts for checking completed cases against requested targets.
- Example generated outputs inside `testing_map/cases/`.

If you are new to the project, the mental model is:

1. Build or use the existing executable.
2. Run one simulation with `testing/_run_win_cpu.bat`.
3. Inspect `.xyz` and `.vtk` outputs.
4. Post-process with the Python scripts.
5. Scale up to parameter studies with `testing_map/run_taguchi_matrix.py`.
6. Verify finished studies with `testing_map/run_verify_taguchi_cases.bat`.

2 Repository Map

Path	What it is for
`README.md`	Short project overview with a basic feature list and workflow summary.
`src/coax_pack_cpu.cpp`	Main solver source. This is the most important file in the repository.
`src/_compile_win.bat`	Windows batch file for compiling the solver with MSVC.
`src/old/`	Older solver versions kept for history and comparison. Usually not the first place to edit.
`testing/_run_win_cpu.bat`	Main single-run driver script for Windows. Best entry point for day-to-day use.
`testing/_fill_stats.py`	Computes radial and axial summary statistics from one `atoms_<iter>.xyz` file.
`testing/_atoms_plot_all.py`	Generates visualizations and animations from `atoms_*.xyz` frames.
`testing/_clean.bat`	Removes common output files from the `testing/` directory.
`testing_map/run_taguchi_matrix.py`	Creates case folders from a CSV design matrix and can optionally execute them.
`testing_map/verify_taguchi_cases.py`	Checks completed Taguchi case folders against requested packing fraction, composition, and diameter-scale targets.
`testing_map/run_verify_taguchi_cases.bat`	Windows wrapper for the verification script.
`testing_map/taguchi_map.csv`	Input CSV for multi-case sweeps.

Path	What it is for
`testing_map/cases/`	Generated study folders with per-case commands, outputs, and manifests.
`testing_map/verification_report.csv`	Generated verifier report with per-case pass/fail status and observed metrics.
`testing_map/success_xyz/`	Generated folder containing the final XYZ file from each passing case.
`testing_map/_fill_stats.py`	Same role as the script in `testing/`, but used inside the sweep workflow.
`testing_map/_atoms_plot_all.py`	Same role as the script in `testing/`, but used inside the sweep workflow.

3 What the Solver Actually Simulates

3.1 Geometry

The physical domain is an annulus:

- Inner cylinder radius: `Rin`
- Outer cylinder radius: `Rout`
- Height: `L`

Particles are injected near the top and then evolve in full 3D Cartesian coordinates (x, y, z).

The domain volume used for the packing fraction estimate is:

$$V_{\text{domain}} = \pi(R_{\text{out}}^2 - R_{\text{in}}^2)L$$

3.2 Particles

Each particle stores:

- position,
- velocity,
- acceleration,
- radius,
- mass,
- a material type identifier,
- sleep-state information for performance.

There are four material types in the current code, corresponding to La, Sr, Fe, and Co. Each type has its own particle size distribution, and optional per-type density values can be used when computing mass.

3.3 Core Physics Model

The solver combines several mechanisms:

- Hard-sphere style particle-particle contact with restitution.
- Tangential damping during contact.
- Inner-wall, outer-wall, bottom-wall, and optional top-piston constraints.
- Optional wall spring-damper penalty response.
- Optional shaking of the container.
- Optional short-range repulsive forcing to encourage spreading.
- Global linear damping.
- Sleep and wake logic to reduce work once particles settle.

This is not a full continuum or bonded-particle model. It is a particle packing and settling simulator designed to be fast enough for parametric studies.

4 How the Main Time Loop Works

At a high level, `src/coax_pack_cpu.cpp` does the following each time step:

1. Inject new particles while the fill stage is active.
2. Rebuild the linked-cell neighbor grid.
3. Wake particles if the ram is active or local conditions require it.
4. Resolve particle-particle contacts.
5. Compute body accelerations from gravity and shaking.
6. Apply optional short-range repulsive accelerations.
7. Integrate particle motion.
8. Apply wall and piston constraints.
9. Update sleep state.
10. Write output files at requested intervals.
11. Check whether the target packing fraction or early-stop criteria have been reached.

This order matters. In the current code, the fill loop now includes an adaptive composition controller by default, so type selection can react to the instantaneous per-species injected volume while the bed is being filled.

5 Composition and Packing Fraction

5.1 Species Fractions

The species fractions are stored as:

```
g_type_frac[4] = {La, Sr, Fe, Co}
```

These fractions are normalized once at startup. In the current solver, they are used as requested composition targets for the adaptive composition controller unless that controller is explicitly disabled.

5.2 Important Interpretation

The code now has two composition modes:

- `-adaptive_composition 1` (default): the solver tracks injected occupied volume by species and biases future injections toward the species that is currently below its requested target.
- `-adaptive_composition 0`: the solver falls back to the legacy fixed-probability inlet sampler.

That means:

- With adaptive control enabled, the code **does** track per-species injected occupied volume and update type selection during filling.
- When `phi_target` is active, the controller steers toward the final requested occupied-volume targets implied by `phi_target` and `g_type_frac[]`.
- When `phi_target` is disabled, the controller still tries to keep the running injected mixture close to the requested fractions.
- The code **does** compute a total packing fraction estimate from the total injected particle volume divided by the annular domain volume.

Even with adaptive control, the final composition is still not guaranteed to be exact. The process remains stochastic, particle sizes differ by species, and the run ends with finite particle counts. In practice, the adaptive controller should be viewed as a steering mechanism, and the verification scripts should be used to confirm whether a case met its requested tolerances.

5.3 Packing Fraction Target

The main packing fraction parameter is `phi_target`. When the estimated total packing fraction reaches this target:

- injection is disabled,
- gravity is switched to zero,
- the ram motion is frozen,
- particles are woken once to allow relaxation.

After that, the run can continue until `niter` or stop earlier if the optional settling criteria are enabled.

6 Important Solver Parameters

This section explains the parameters a student will most often change. The flag-based interface is the recommended one.

6.1 Always Important

Flag	Typical type	Meaning
`-natoms_max`	integer	Maximum number of particles that may be injected.
`-dt`	real	Time step in seconds. Smaller is more stable but slower.
`-niter`	integer	Total number of time steps.
`-dump_interval`	integer	Base output cadence.
`-debug`	0 or 1	Enables extra console messages.
`-seed`	integer	Random number seed.
`-threads`	integer	OpenMP thread count. Zero means automatic.
`-rin`	real	Inner radius of annulus in meters.
`-rout`	real	Outer radius of annulus in meters.
`-length`	real	Column length in meters.
`-flux`	integer	Injection rate in particles per second.
`-gravity`	real	Downward gravity magnitude.
`-fill_time`	real	Duration of the injection stage.
`-phi_target`	real	Stop-filling threshold based on estimated packed volume fraction.

6.2 Composition and Size

Flag	Typical type	Meaning
`-f_la`	real	Requested La occupied-volume fraction target.
`-f_sr`	real	Requested Sr occupied-volume fraction target.
`-f_fe`	real	Requested Fe occupied-volume fraction target.
`-f_co`	real	Requested Co occupied-volume fraction target.
`-diameter_scale_factor`	real	Multiplies all sampled diameters. Useful for sensitivity studies.
`-adaptive_composition`	0 or 1	Enables the feedback controller that updates type selection from instantaneous per-species injected volume.

6.3 Shaking and Motion

Flag	Typical type	Meaning
`-shake_hz`	real	Shaking frequency.
`-shake_amp`	real	Legacy vertical shake amplitude.
`-shake_amp_x`	real	Independent x-axis shake amplitude.
`-shake_amp_y`	real	Independent y-axis shake amplitude.
`-shake_amp_z`	real	Independent z-axis shake amplitude.
`-shake_xy_legacy`	0 or 1	If 1, historical vertical shaking also affects x and y.
`-inject_vx`	real	Initial x velocity of injected particles.
`-inject_vy`	real	Initial y velocity of injected particles.

Flag	Typical type	Meaning
`-inject_vz`	real	Initial z velocity of injected particles. Usually negative.

6.4 Contacts, Damping, and Stabilization

Flag	Typical type	Meaning
`-cushion`	real	Expands effective collision radius without changing physical radius in output.
`-lin_damp`	real	Global linear velocity damping coefficient.
`-e_pp`	real	Restitution coefficient for particle-particle contact.
`-e_pw`	real	Restitution coefficient for particle-wall contact.
`-tangent_damp`	real	Tangential damping factor during contact.
`-wall_k`	real	Wall spring stiffness. Zero disables it.
`-wall_zeta`	real	Wall damping ratio.
`-wall_dvmax`	real	Velocity increment cap from wall spring response.
`-wall_rough_amp`	real	Wall roughness amplitude.
`-wall_rough_mth`	integer	Angular roughness mode count.
`-wall_rough_mz`	integer	Axial roughness mode count.

6.5 Repulsion and Settling Controls

Flag	Typical type	Meaning
`-repulse_range`	real	Gap range over which repulsion acts. Zero disables repulsion.
`-repulse_k_pp`	real	Particle-particle repulsion strength.
`-repulse_k_pw`	real	Particle-wall repulsion strength.
`-repulse_use_mass`	0 or 1	Switches interpretation of repulsion coefficient.
`-repulse_dvmax`	real	Velocity increment cap from repulsion.
`-ram_start`	real	Time when top-ram motion begins.
`-ram_duration`	real	Duration of the ram stage.
`-ram_speed`	real	Downward ram speed.
`-stop_vrms`	real	Optional RMS-velocity settling threshold after `phi_target`.
`-stop_vmax`	real	Optional max-velocity settling threshold after `phi_target`.
`-stop_sleep_frac`	real	Optional required fraction of sleeping particles.
`-stop_check_interval`	integer	How often to evaluate stop conditions.
`-stop_checks_required`	integer	Number of consecutive successful stop checks required.

6.6 Output Controls

Flag	Typical type	Meaning
`-xyz_interval`	integer	XYZ output interval. If negative, follows `dump_interval`.
`-vtk_interval`	integer	Particle VTK output interval. If negative, follows `dump_interval`.
`-vtk_domain_interval`	integer	Output interval for domain surface VTK files. Zero disables them.
`-vtk_domain_segments`	integer	Number of segments used to tessellate the annular geometry in VTK.

7 How to Build the Solver

7.1 Windows

The repository already includes `src/_compile_win.bat`. It expects a local Visual Studio Build Tools installation and calls `vcvarsall.bat` before compiling.

The command used there is approximately:

```
cl /EHsc /O2 /openmp:llvm /std:c++17 /Fe:coax_pack_cpu.exe coax_pack_cpu.cpp
```

If the executable already exists in `src/`, you may not need to rebuild immediately. Still, if you edit `src/coax_pack_cpu.cpp`, rebuild before running new cases.

7.2 Linux or macOS

The `README.md` suggests a GCC-style build such as:

```
g++ -O3 -march=native -fopenmp -std=c++17 -o coax_pack_cpu coax_pack_cpu.cpp
```

This repository is clearly organized around Windows batch workflows, so graduate students on Linux should expect to adapt the run scripts manually.

8 How to Run One Simulation

The easiest entry point is:

`testing/_run_win_cpu.bat`

That script:

- defines a simulation parameter set,
- cleans previous outputs in `testing/`,
- runs `..\src\coax_pack_cpu.exe`,
- optionally runs statistics and plotting scripts afterward.

8.1 Recommended First Run

Before doing a large study, try this sequence:

1. Open `testing/_run_win_cpu.bat`.
2. Reduce `NATOMS_MAX` and `NITER` if you want a quick smoke test.
3. Keep `VF`, geometry, and output intervals simple.
4. Run the batch file from Windows.
5. Verify that `atoms_*.xyz` and `atom.*.vtk` files appear in `testing/`.

8.2 Why This Matters

This first run checks three things at once:

- the executable works,
- the output paths are correct,
- your Python post-processing environment is at least partially usable.

9 Understanding the Output Files

9.1 XYZ Files

The solver writes files such as:

`atoms_1000.xyz`

These contain one particle per line and include:

- element symbol,
- x, y, and z position,
- particle diameter, written as $2r$.

These files are especially useful for:

- lightweight analysis in Python,
- custom statistics,
- simple debugging of geometry and fill behavior.

9.2 VTK Files

The solver also writes particle VTK files such as:

`atom.1000.vtk`

These are intended for ParaView. The VTK files include particle positions and scalar fields such as radius and type ID, plus a string field for element symbol.

9.3 Optional Domain VTK

If `-vtk_domain_interval` is enabled, the code can also write a geometry surface VTK file for the annular domain and top boundary. This helps visually compare particles to the confining geometry.

10 Post-Processing Scripts

10.1 `testing/_fill_stats.py`

This script reads one XYZ snapshot and computes:

- radial mean particle radius,
- radial coefficient of variation,
- a radial segregation index,
- axial mean particle radius,
- axial coefficient of variation,
- a 2D heatmap in (r, z),
- a CSV summary table.

The solver writes particle diameter in the fourth numeric XYZ column. The statistics scripts use that value directly as `a`, so if you need a physically exact radius-based interpretation, review the script and convert $a \to a/2$ where appropriate before using the results quantitatively.

10.2 Interpretation Caution

This script is mainly radius-based. It does not directly compute exact species-wise volume fractions or full bed porosity. If you need those, you would write an additional analysis script.

10.3 `testing/_atoms_plot_all.py`

This script builds visualizations and animations from a sequence of XYZ frames. It is useful for quickly checking:

- whether injection is happening where you expect,
- whether particles are piling up or segregating,
- whether shaking or repulsion changes the bed structure over time.

10.4 `testing_map/verify_taguchi_cases.py`

This script checks finished Taguchi cases against the requested study design. For each case folder, it:

- verifies the generated case metadata matches the row in `taguchi_map.csv`,
- finds the final `atoms_*.xyz` snapshot,
- computes the observed total packing fraction,

- computes observed species occupied-volume fractions,
- infers the realized diameter scale factor,
- writes a pass/fail report.

The Windows wrapper `testing_map/run_verify_taguchi_cases.bat` is the easiest way to launch it after a sweep is complete.

11 Taguchi and Multi-Case Workflow

11.1 Why This Exists

Once a single simulation works, researchers usually want parameter sweeps. The repository supports that with:

`testing_map/run_taguchi_matrix.py`

This script reads a CSV and creates one folder per case.

11.2 Expected CSV Columns

The current script expects at least:

```
Run, diameter_scale_factor, PF_value, f_La, f_Sr, f_Fe, f_Co
```

11.3 What the Script Produces

For each case, the script writes:

- `case_params.json`,
- `command.txt`,
- `run_case.bat`.

It also writes:

- a study-level `manifest.csv`,
- a study-level `run_all_cases.bat`.

11.4 How to Think About a Case Folder

Each case folder is meant to be self-contained enough that you can:

- see exactly which flags were used,
- rerun the case,
- inspect outputs without mixing them with other runs.

11.5 Suggested Workflow for Sweeps

1. Edit `testing_map/taguchi_map.csv`.
2. Run the case generator.
3. Review one or two generated `command.txt` files before launching everything.
4. Run cases sequentially with `run_all_cases.bat` or let the Python driver execute them.
5. Run `run_verify_taguchi_cases.bat` after the sweep completes.
6. Use `manifest.csv` and `verification_report.csv` together as the study ledger.

12 Old Versions and Historical Files

The `src/old/` and `testing/old/` folders contain older development snapshots. These are useful when:

- you want to compare behavior across solver revisions,
- you are searching for when a feature was introduced,
- you need to recover an older implementation idea.

They are usually **not** the right place for routine edits unless you are doing deliberate archaeology.

13 Practical Advice for New Graduate Students

13.1 Start Small

When you inherit a DEM codebase, the biggest risk is changing too many things before you know what a healthy run looks like. Start with:

- fewer particles,
- shorter runs,
- frequent output,
- one changed parameter at a time.

13.2 Record Assumptions

For every study, keep a notebook or spreadsheet that records:

- the git commit or repository snapshot,
- which batch or Python script launched the run,
- the exact parameter changes from baseline,
- what outcome you expected before the run.

13.3 Common Beginner Mistakes

- Assuming adaptive control guarantees exact final packed fractions without post-run verification.
- Running too few time steps after injection ends and then calling the structure settled.

- Forgetting that output intervals may be different for XYZ and VTK.
- Mixing results from different cases in the same folder.
- Changing multiple physics knobs at once and losing interpretability.

14 Suggested First-Week Learning Plan

1. Read `README.md` and this manual once without editing anything.
2. Open `src/coax_pack_cpu.cpp` and skim the global parameters and the main loop.
3. Run one tiny case with `testing/_run_win_cpu.bat`.
4. Open one `atom.*.vtk` file in ParaView.
5. Run `_fill_stats.py` on one final XYZ frame.
6. Generate two or three Taguchi cases and inspect the generated `command.txt` files.
7. Run the verification script on those pilot cases.
8. Only after that, begin changing physics or composition settings.

15 Where to Edit for Common Tasks

Goal	**Where to start**
Change default solver behavior	`src/coax_pack_cpu.cpp`
Compile on Windows	`src/_compile_win.bat`
Run a single case	`testing/_run_win_cpu.bat`
Add a simple new post-processing metric	`testing/_fill_stats.py`
Automate a parameter sweep	`testing_map/run_taguchi_matrix.py`
Verify a completed sweep	`testing_map/verify_taguchi_cases.py`
Adjust study design values	`testing_map/taguchi_map.csv`
Compare against an older solver revision	`src/old/`

16 Known Limits of the Current Repository

Based on the current source tree, new users should be aware of these limits:

- The project is strongly Windows-oriented in its automation.
- Adaptive composition control improves agreement with requested targets, but it still does not guarantee an exact final composition.
- Post-processing is helpful but still fairly lightweight; some research questions will need custom scripts.
- Many generated outputs are large, so storage discipline matters.
- Old generated case folders inside `testing_map/cases/` can make the repository look much larger than the hand-written source actually is.

17 Final Takeaway

The heart of this repository is simple:

- `src/coax_pack_cpu.cpp` is the simulator.
- `testing/` is for running and inspecting one case.
- `testing_map/` is for systematic multi-case studies.

If you understand those three layers, you can already do productive research with this codebase.